

Ejercicios Prácticos (Anexo) - Unidad 3, Sesión 2

Introducción a LangGraph

Anexo a [ejercicios.md](#): este material complementa los ejercicios principales con un único **ejercicio guiado** centrado en **LangGraph**. Se desarrolla en 4 etapas incrementales: chatbot mínimo → memoria con checkpointer → enrutado condicional → agente ReAct con tool personalizada.

Configuración Previa

Estos ejercicios asumen que ya tienes configurado el entorno de la sesión (variables `OPENAI_API_KEY` o `OPENROUTER_API_KEY` en el `.env`). Solo necesitas instalar los paquetes adicionales de LangGraph:

```
pip install langgraph langchain-openai python-dotenv
```

Alternativa gratuita con OpenRouter: todos los ejemplos siguientes usan `ChatOpenAI` de LangChain. Para usar OpenRouter en lugar de OpenAI, sustituye la inicialización del modelo por:

```
import os
modelo = ChatOpenAI(
    model="google/gemini-2.0-flash-exp:free",
    temperature=0,
    openai_api_key=os.getenv("OPENROUTER_API_KEY"),
    openai_api_base="https://openrouter.ai/api/v1",
)
```

Ejercicio Guiado: Construyendo un Asistente con LangGraph

Metadata

- **Duración estimada:** 60 minutos (4 etapas de ~15 min)
- **Tipo:** Programación guiada incremental
- **Modalidad:** Individual
- **Dificultad:** Intermedia
- **Prerequisitos:**
 - Ejercicios 1-5 de [ejercicios.md](#) completados
 - Lectura de [teoria2.md](#)

Contexto

Vas a construir un asistente conversacional usando LangGraph, partiendo del grafo más simple posible y añadiendo capacidades en cada etapa. El objetivo no es el producto final, sino **interiorizar el modelo**

mental de grafos de estado y comparar cada paso con lo que harías en una Chain LCEL clásica.

Objetivo de Aprendizaje

- Construir un `StateGraph` con `MessagesState`, nodos y edges
- Añadir memoria persistente con `MemorySaver` y `thread_id`
- Implementar un edge condicional con función router
- Definir tools con `@tool` y construir un agente con `create_react_agent`
- Visualizar el grafo resultante con `draw_ascii()`

Etapas 1: Chatbot Mínimo (15 min)

Implementa el chatbot más sencillo posible: un grafo con un único nodo que llama al modelo.

Archivo: `etapa1_chatbot.py`

Completa las partes marcadas con `# TODO`:

```
import os
from dotenv import load_dotenv
from langgraph.graph import StateGraph, MessagesState, START, END
from langchain_openai import ChatOpenAI

load_dotenv()

# Modelo (sustituir por OpenRouter si no se tiene OpenAI)
modelo = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)

# TODO 1: Define un nodo llamado `llamar_modelo` que:
# - reciba `state: MessagesState`
# - llame a modelo.invoke(state["messages"])
# - devuelva un dict con la clave "messages" conteniendo la respuesta
def llamar_modelo(state: MessagesState) -> dict:
    ____

# TODO 2: Construye el grafo
# - crea un StateGraph(MessagesState)
# - añade el nodo "chatbot" con la función llamar_modelo
# - conecta START -> "chatbot" -> END
builder = ____
____
____

# TODO 3: Compila el grafo
graph = ____

# TODO 4: Visualiza el grafo en ASCII
print(graph.get_graph().draw_ascii())

# TODO 5: Invoca el grafo con la pregunta "¿Qué es Python?"
```

```

resultado = ____

# Imprimir la respuesta del modelo (último mensaje del historial)
print(resultado["messages"][-1].content)

```

Verificación

Al ejecutar tu script deberías ver:

1. Un diagrama ASCII del grafo (3 nodos: `__start__`, `chatbot`, `__end__`).
2. Una respuesta del modelo explicando qué es Python.

Preguntas de Reflexión

1. ¿Cuántas líneas de código requiere este chatbot frente a `prompt | modelo` con LCEL?
2. ¿Qué tipo tiene el último elemento de `resultado["messages"]`? Imprime `type(resultado["messages"][-1])`.
3. ¿Qué pasaría si el nodo devolviera `{"messages": resultado_completo}` en vez de `{"messages": [respuesta]}`? Pruébalo.

Etapas 2: Añadir Memoria con `MemorySaver` (15 min)

Ahora vamos a darle memoria al chatbot **sin escribir lógica de gestión de historial**, simplemente añadiendo un checkpointer.

Archivo: `etapa2_memoria.py`

Parte del código de la Etapa 1 y añade lo siguiente:

```

import os
from dotenv import load_dotenv
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.checkpoint.memory import MemorySaver
from langchain_openai import ChatOpenAI

load_dotenv()

modelo = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)

def llamar_modelo(state: MessagesState) -> dict:
    return {"messages": [modelo.invoke(state["messages"])]}

builder = StateGraph(MessagesState)
builder.add_node("chatbot", llamar_modelo)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

# TODO 1: Crea un MemorySaver y compila el grafo CON ese checkpointer
checkpointer = ____

```

```

graph = builder.compile(checkpointer=____)

# TODO 2: Define la configuración con un thread_id (puede ser cualquier
string)
config = {"configurable": {"thread_id": ____}}

# TODO 3: Bucle interactivo de conversación
# - leer input del usuario con input("Tú: ")
# - si escribe "salir", terminar
# - en otro caso, invocar graph con config y mostrar la respuesta
print("Chatbot con memoria. Escribe 'salir' para terminar.\n")
while True:
    user_msg = input("Tú: ").strip()
    if ____:
        break
    if not user_msg:
        continue

    resultado = graph.invoke(
        {"messages": [{"role": "user", "content": ____}]},
        config=____,
    )
    print(f"Bot: {resultado['messages'][-1].content}\n")

# TODO 4 (opcional): después de salir del bucle, imprime cuántos mensajes
# tiene el historial en el thread, usando graph.get_state(config)
estado = graph.get_state(config)
print(f"\nMensajes guardados en el thread:
{len(estado.values['messages'])}")

```

Verificación: Conversación de Prueba

Mantén esta conversación y comprueba que el bot **recuerda**:

1. Tú: Hola, me llamo [tu nombre]
2. Tú: Estoy aprendiendo LangGraph
3. Tú: ¿Cómo me llamo? ← debe responder con tu nombre
4. Tú: ¿Sobre qué tema dije que estoy aprendiendo? ← debe decir LangGraph

Reto Adicional

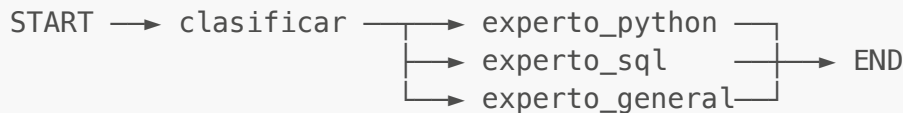
- Cambia el `thread_id` a otro valor distinto y reinicia la conversación. ¿El bot recuerda lo anterior? Justifica por qué.
- Reemplaza `MemorySaver()` por `SqliteSaver.from_conn_string("chat.db")` (necesita `pip install langgraph-checkpoint-sqlite`). Reinicia el script entero y comprueba si el historial sobrevive entre ejecuciones.

Preguntas de Reflexión

1. ¿Cuánto código de gestión de historial has escrito tú? Compara con el chatbot manual del Ejercicio 3 de [ejercicios.md](#).
2. ¿Qué representan los `thread_id` en una aplicación real (ej. WhatsApp, Slack, web)?
3. ¿Cuándo elegirías `MemorySaver` vs `SqliteSaver` vs `PostgresSaver`?

Etapla 3: Enrutado Condicional - Asistente con Especialidades (15 min)

Vamos a construir un asistente que **clasifica** la pregunta del usuario y la enruta a un experto distinto según el tema.



Archivo: `etapa3_router.py`

```

import os
from typing import Annotated, TypedDict, Literal
from dotenv import load_dotenv
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_openai import ChatOpenAI

load_dotenv()
modelo = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# TODO 1: Define un TypedDict EstadoRouter con:
# - messages: Annotated[list, add_messages]
# - categoria: str
class EstadoRouter(TypedDict):
    __slots__ = ()

# --- Nodo clasificador ---
def clasificar(state: EstadoRouter) -> dict:
    pregunta = state["messages"][-1].content
    prompt = (
        "Clasifica la siguiente pregunta en UNA SOLA palabra entre: "
        "'python', 'sql' o 'general'. Responde SOLO con esa palabra.\n\n"
        f"Pregunta: {pregunta}"
    )
    salida = modelo.invoke(prompt).content.strip().lower()

    # Normalizar la salida (puede contener basura)
    if "python" in salida:
        cat = "python"
    elif "sql" in salida:
        cat = "sql"
  
```

```

    else:
        cat = "general"

    return {"categoria": cat}

# --- TODO 2: Define tres nodos especialistas ---
# Cada uno recibe el estado, llama al modelo con un system prompt diferente
# y devuelve {"messages": [respuesta]}.

def experto_python(state: EstadoRouter) -> dict:
    system = "Eres un experto en Python. Responde con código limpio y comentarios."
    pregunta = state["messages"][-1].content
    respuesta = modelo.invoke([
        {"role": "system", "content": system},
        {"role": "user", "content": pregunta},
    ])
    return {"messages": [respuesta]}

def experto_sql(state: EstadoRouter) -> dict:
    # TODO: copia el patrón de experto_python con un system prompt sobre SQL
    _____

def experto_general(state: EstadoRouter) -> dict:
    # TODO: copia el patrón con un system prompt de asistente generalista
    _____

# --- TODO 3: Define la función router ---
# Recibe el estado y devuelve el nombre del nodo siguiente
# según state["categoria"].
def decidir_ruta(state: EstadoRouter) -> Literal["experto_python",
"experto_sql", "experto_general"]:
    _____

# --- Construir el grafo ---
builder = StateGraph(EstadoRouter)
builder.add_node("clasificar", clasificar)
builder.add_node("experto_python", experto_python)
builder.add_node("experto_sql", experto_sql)
builder.add_node("experto_general", experto_general)

builder.add_edge(START, "clasificar")

# TODO 4: Añade el edge condicional desde "clasificar" usando decidir_ruta
builder.add_conditional_edges(_____, _____)

# TODO 5: Conecta cada experto a END
_____
_____
_____

graph = builder.compile()

```

```
# Visualizar el grafo
print(graph.get_graph().draw_ascii())

# --- Probar con tres preguntas distintas ---
preguntas = [
    "¿Cómo escribo una list comprehension?",
    "¿Cómo hago un LEFT JOIN entre dos tablas?",
    "¿Cuál es la capital de Francia?",
]

for p in preguntas:
    print("\n" + "=" * 60)
    print(f"Pregunta: {p}")
    resultado = graph.invoke({"messages": [{"role": "user", "content":
p}]}))
    print(f"Categoría detectada: {resultado['categoria']}")
    print(f"Respuesta: {resultado['messages'][-1].content[:200]}...")
```

Verificación

Al ejecutar deberías obtener:

- La pregunta de Python clasificada como **python** y respondida con código.
- La de SQL clasificada como **sql** y respondida con una sentencia **SELECT**.
- La de geografía clasificada como **general** y respondida sin código.

Reto Adicional

Añade un cuarto experto, **experto_traduccion**, que se active si la pregunta contiene la palabra "traduce" o "translate". Modifica **clasificar** y **decidir_ruta** para soportarlo.

Preguntas de Reflexión

1. ¿Por qué **clasificar** devuelve **{"categoria": cat}** y no **{"messages": [...]}**? ¿Qué cambia en el estado tras ese nodo?
2. ¿Sería posible hacer este enrutado con una Chain LCEL (**prompt | modelo | parser**)? ¿Qué problema encuentras?
3. ¿Qué ocurre si el clasificador devuelve una categoría inesperada (ej. "ruby")? ¿Cómo blindarías el router?

Etapas 4: Agente ReAct con Tool Personalizada (15 min)

Vamos a saltar al patrón estrella de LangGraph: un agente que **decide** cuándo usar herramientas. Este es un anticipo de la **Unidad 4**.

Archivo: **etapa4_agente.py**

```

import os
import math
from datetime import datetime
from dotenv import load_dotenv
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent
from langgraph.checkpoint.memory import MemorySaver

load_dotenv()

# --- TODO 1: Define 3 tools con el decorador @tool ---
# Cada tool DEBE tener un docstring claro: el LLM lo usará para decidir
# cuándo invocarla.

@tool
def calcular(expresion: str) -> str:
    """Evalúa una expresión matemática segura (suma, resta, mult, división,
    potencias, funciones de math). Ejemplo: 'sqrt(144) + 2*3'."""
    try:
        # eval restringido: solo nombres del módulo math + operadores
        permitidos = {k: getattr(math, k) for k in dir(math) if not
        k.startswith("__")}
        return str(eval(expresion, {"__builtins__": {}}, permitidos))
    except Exception as e:
        return f"Error: {e}"

# TODO: completa la siguiente tool. Debe devolver la fecha y hora actuales
# como string en formato "YYYY-MM-DD HH:MM".
@tool
def hora_actual() -> str:
    """Devuelve la fecha y hora actuales del sistema."""
    _____

# TODO: completa esta tool. Recibe un string `ciudad` y devuelve un string
# con el clima simulado (puedes hardcodear un dict de 3-4 ciudades).
@tool
def clima(ciudad: str) -> str:
    """Devuelve el clima actual de una ciudad española (datos
    simulados)."""
    _____

tools = [calcular, hora_actual, clima]

# --- TODO 2: Crear el agente ReAct con memoria ---
modelo = ChatOpenAI(model="gpt-4o-mini")
agente = create_react_agent(
    _____,
    tools=_____,
    checkpoint=_____,
)

```

```
# Visualizar
print(agente.get_graph().draw_ascii())

# --- TODO 3: Conversación de prueba ---
config = {"configurable": {"thread_id": "alumno_1"}}

preguntas = [
    "¿Qué hora es y cuánto es la raíz cuadrada de 256?",
    "¿Qué tiempo hace en Madrid?",
    "¿Y en Bilbao?",    # debe entender que sigue hablando del clima
]

for p in preguntas:
    print("\n" + "=" * 60)
    print(f"Usuario: {p}")
    respuesta = agente.invoke(
        {"messages": [{"role": "user", "content": p}]},
        config=config,
    )
    # El último mensaje es la respuesta final del agente
    print(f"\nAgente: {respuesta['messages'][-1].content}")

    # TODO 4 (opcional): imprime los mensajes intermedios para ver el
    "razonamiento"
    # print("\n--- Trazas internas ---")
    # for m in respuesta["messages"]:
    #     m.pretty_print()
```

Verificación

Al ejecutar deberías observar:

1. La 1ª pregunta dispara **dos** llamadas a tools (**hora_actual** y **calcular**). Habilita el bloque de "Trazas internas" para verlo.
2. La 2ª pregunta dispara **clima("Madrid")**.
3. La 3ª pregunta solo dice "Bilbao": el agente debe inferir gracias a la memoria que el usuario sigue preguntando por el tiempo.

Reto Adicional

- Añade una tool **buscar_wikipedia(termino: str)** que use **wikipedia** (**pip install wikipedia**) para devolver el primer párrafo de un artículo. Pregunta al agente algo como "*¿Quién fue Ada Lovelace?*" y observa cómo decide usar la nueva tool.
- Cambia **ChatOpenAI** por **ChatAnthropic** (Claude) y verifica que el mismo agente funciona sin cambios adicionales.

Preguntas de Reflexión

1. ¿Qué pasa si quitas el docstring de la tool **clima**? ¿Por qué la calidad de las decisiones del agente depende de los docstrings?

2. Compara el código de este agente (~30 líneas) con el ejemplo de Function Calling manual de la teoría principal (sección 4.4). ¿Qué le añade `create_react_agent` que tú tendrías que escribir tú mismo?
3. ¿En qué situaciones reales sería útil este patrón (agente con tools)? Da 3 ejemplos.

Soluciones de Referencia

- ▶ Ver solución Etapa 1 - Chatbot Mínimo
 - ▶ Ver solución Etapa 2 - Memoria
 - ▶ Ver solución Etapa 3 - Router
 - ▶ Ver solución Etapa 4 - Agente ReAct
-

Entregable

Sube a tu repositorio del curso los **cuatro archivos** generados:

```
unidad_3/  
├── sesion_2/  
│   └── langgraph/  
│       ├── etapa1_chatbot.py  
│       ├── etapa2_memoria.py  
│       ├── etapa3_router.py  
│       └── etapa4_agente.py
```

Acompaña el código con un breve `README.md` (máximo media página) respondiendo a:

1. ¿En cuál de las 4 etapas viste más claramente la ventaja de LangGraph sobre una Chain LCEL?
2. ¿Cuál fue el bug o malentendido más difícil de depurar mientras hacías los ejercicios?
3. Diseña (sin programarlo) un grafo para una aplicación real de tu interés. Describe los nodos, las aristas y el estado en 5-10 líneas.